

BoilerBridge: Sprint 3 Retrospective

Team 26 - BoilerBridge: Xavion Marable, Logan Wilkinson, Emilio Guzman,
Shashank Mukkera, Shreyansh Tehanguria

1. What went well? (0.5 point)

- a. You may write this section in sentences and/or list successful user stories and tasks with detailed discussions.
- Group Voting (User Story #1) was implemented with a Mongoose vote schema, backend POST and DELETE routes, and a voting UI on individual itinerary cards that prevents duplicate votes per user per activity. Member Removal (User Story #2) was completed with leader-only authorization and a confirmation modal that prevents accidental removals. Drag and Drop Reordering (User Story #3) was integrated with display order persisted to the database on drop and a read-only tooltip shown to Viewers. Lockdown Mode (User Story #4) was implemented with an isLocked field on the CalendarEvent model, a toggleable lock icon on each activity card, an unlock confirmation dialog, and generator logic that skips locked activities during regeneration.
- Public Itinerary Feed (User Story #15) was implemented smoothly. Successfully added backend support for publishing itineraries, visibility controls, and a public feed where users can browse itineraries created by others. UI cards with itinerary previews and detailed viewing functionality were completed. Partial Itinerary Editing (User Story #16) also progressed well. Section-level editing allowed users to modify only a single activity or day without affecting the rest of the itinerary. Persistence after refresh and selective

updates worked as expected, making the feature practical and user-friendly.

- Google Calendar Sync (User Story #5) was implemented end-to-end with a CalendarConnection Mongoose schema, AES-256-GCM token encryption, a full OAuth flow for Google account linking, CRUD endpoints for managing connections, manual sync via a dedicated button, and duplicate-prevention through a CalendarEventSync mapping schema. A 22-test integration suite covering auth, connection management, sync behavior, and duplicate prevention was written and passing. Itinerary Map (User Story #6) was delivered with an ItineraryStop Mongoose schema, GET/POST/PUT/DELETE API routes with membership and permission checks, and an ItineraryMapPanel component that automatically geocodes event location strings from the itinerary, renders numbered markers on a Google Map, draws an amber polyline connecting stops in chronological order, and displays an info window with stop details on click. A 25-test integration suite covering auth, ordering, coordinate validation, and permission rules was written and passing. Both features were integrated into the group dashboard as new tabs.
- The review and implementation changes needed for writing user reviews (User Story #7) and it turned out well. The already partially implemented reviews backend and frontend was a huge headstart in the process and gave me something to build off of making the implementation process smooth. As well as this, adding the functionality for sharing itineraries (User Story #8) went smoothly and without a hitch. Users have the ability to toggle sharing via link as well as a copy link button to share it with friends. The 10-day weather forecast (User Story #9) in the itinerary view, was also implemented smoothly and now users can see forecasts for their destinations. The in-app travel notifications (User Story #10) weren't too hard to implement, piggy backing off of old settings code written, making the addition of in-app notification settings not take as long as expected and now allows users to now toggle both email and in-app notifications.

2. What did not go well? (0.5 point)

- a. Include general retrospective review for this sprint.
 - b. Ensure to list ALL unsuccessful user stories and tasks with detailed discussions. (They should be in line with the Sprint Planning Document for the next sprint.)
- The sprint planning document contained duplicate user stories between two team members — Group Voting (User Story #1) and Vote on Activity Options (User Story #12) covered overlapping functionality, which caused confusion around ownership and scope. A version control issue also caused lock-related changes to be overwritten during a branch merge late in the sprint, requiring manual restoration from a prior commit.
 - Offline Itinerary View (User Story #17) created confusion during testing. On my local machine, browser-related issues and an outdated environment made offline refresh behavior inconsistent, which led to spending a significant amount of time debugging whether the issue was code-related or browser-related.
 - Outlook Calendar integration (User Story #5, AC1, AC2, AC3, AC6, AC7, AC8) could not be completed because Microsoft Azure App Registration requires a paid organizational account that was not available, leaving all Outlook-related acceptance criteria unmet. Auto-sync on trip date changes (User Story #5, AC2) was not implemented because Google OAuth apps in testing mode issue refresh tokens that expire after 7 days, making a reliable background sync trigger incompatible with the current development environment — manual sync was provided as an alternative. Merge conflicts in `page.tsx`, `jest.setup.js`, and `Group.tsx` caused delays on both User Story #5 and User Story #6 after pulling from main mid-sprint. A Node version mismatch between the local environment (Node 25) and the project requirement (Node 22) blocked test writing for both User Story #5 and User Story #6 until resolved with `nvm`.

3. How should you improve? (1.0 point)

- User stories should be reviewed as a team before the sprint begins to catch duplicate or overlapping work early. Improving communication throughout the sprint would help surface integration issues before they compound. Better coordination around shared files during development would also prevent merge conflicts from appearing late in the sprint. Going into the presentation, making sure all features are stable and demonstrable on the main branch should be the priority.
- Third-party service requirements should be verified before sprint planning, as the Outlook integration for User Story #5 was committed to without confirming that Azure App Registration had a free tier, resulting in undeliverable acceptance criteria. OAuth features that depend on long-lived tokens should account for testing-mode limitations early, since Google's 7-day refresh token expiry in testing mode made auto-sync for User Story #5 unreliable in the development environment. The team should pull from main more frequently throughout the sprint, as waiting until late in the sprint caused the merge conflicts affecting User Story #5 and User Story #6 to be larger and more time-consuming to resolve. Environment and dependency changes on main should be communicated to all team members at the start of each sprint to prevent mid-sprint blockers like the Node version mismatch that delayed both user stories.
- For itinerary generation, instead of relying only on a locally hosted Ollama model, the team could explore stronger free or low-cost API-based open-source models with better reasoning and larger parameter sizes. This could improve itinerary quality, reduce logical errors in schedules, and provide more reliable outputs for users.
- The option to vote on itinerary generations was implemented independently by two people (user stories 1 and 12). Communication over there could be improved.
- The scope of user story 14 could have been planned out better. I realized midway during

implementation that the API didn't work as expected and had to change the scope during implementation.

4. Notes

- a. For section 1 and 2, only listing out some user stories and tasks without any discussion will not get you full credits.
- b. Please make sure to include your project name, team number and the names of all your team members.

Total: 2 points (2% of your project grade)